

杭州电子科技大学信息工程学院

毕业设计（论文）开题报告

题 目	基于 Unity 的动作游戏与编辑器工具的设计与实现
系	计算机系
专 业	软件工程
姓 名	邬锦飞
班 级	软件工程一班
学 号	259400232
指导教师	王欣

一、综述本课题国内外研究动态，说明选题的依据和意义

1.1 国内研究动态

动作游戏的内容生产依赖连招与判定参数的反复调校。国内游戏公司在动作类项目研发中已普遍采用“有限状态机 + 数据驱动 + 自研编辑器工具链”的生产方式，形成了连招配置表、动作取消窗口、判定帧窗口等做法，战斗内容的生产逐步从代码硬编码转向可视化配置。

国内相关研究多集中于角色动画控制、有限状态机、行为树 AI 等单点技术；以“使用 Unity 实现一款××游戏”为题的毕业设计，编辑器工具多作为附属功能出现。具体到连招配置，公开资料多为商业插件附带的通用技能系统，面向“每段连招的动画片段、判定起止时间、伤害值、连招窗口时间与位移参数”的可视化配置与校验，缺少可复现的设计方案；许多项目还把连招参数直接写在角色控制器里，修改一次参数就要改动代码并重新编译，非程序人员无法参与调优，参数之间的约束关系没有检查手段，错误只能在运行时以“连招接不上”“挥空却打中”等形式暴露。因此，面向动作游戏连招内容的可视化配置工具具有明确的实践价值。

1.2 国外研究动态

国外在引擎生态与工具化方面起步更早。Unity 官方提供了完整的编辑器扩展接口（EditorWindow、CustomEditor、SerializedObject、ReorderableList 等），团队工具可以直接嵌入编辑器工作流。行业实践中，动作游戏的技能与连招编辑器是常见的自研工具，技术社区有相关经验分享，其共同点是把动画、判定与表现做成时间轴上的可视化配置。

在方法层面，《游戏编程模式》论述了组件模式、状态模式与观察者模式的应用，《游戏引擎架构》对角色动画系统、混合树与连续碰撞检测作了完整阐述，二者分别是数据驱动设计与判定时序设计的参考。总体来看，国外方案多面向通用引擎能力与商业插件，配置粒度较粗；面向中小规模项目、聚焦“连招段参数 + 动画时序 + 编辑期校验”的可复现设计仍然较少，这正是本课题的切入点。

1.3 选题依据与意义

本课题的研究对象是 Unity 编辑器扩展工具，具体为面向动作游戏的连招配置编辑器；同时实现一个可运行的动作游戏 Demo，作为验证该工具可用性的验证平台，游戏玩法本身不作为研究内容。

选题依据来自这一工作场景的具体矛盾：连招参数时序性强、相互约束多、出错不报错。以代码硬编码或直接在 Inspector 中编辑的方式组织这类数据存在三个问题：修改成本高，协作门槛高，约束关系无法表达。本课题的意义在于：把主观的连招手感拆解为可度量、可配置的参数；通过“编辑器修改→资产→运行时生效”的链路使连招内容的生产脱离代码修改；把配置合法性的检查前移到编辑期。本课题的重点与特色是：以“连招段”为配置单元，把动画、判定时间区间、伤害、连招窗口与位移参数统一到一套可视化配置中，并由运行时按同一份资产执行。

二、研究的基本内容，拟解决的主要问题

2.1 基本内容

本课题的目标是：设计并实现一个基于 Unity 的动作游戏 Demo，包含角色移动、攻击连招、受击反馈和简单敌人 AI 等基础动作游戏要素；同时开发一个 Unity 编辑器扩展工具——连招配置编辑器，使策划与开发者能够通过可视化界面配置连招转换规则、攻击判定参数与动画事件，无需直接修改代码。

具体内容包括：连招以“段”为单位组织，工具支持连招步骤的添加、删除与排序；每段可配置动画片段、动画事件、攻击判定起始与结束时间、伤害值、连招窗口时间与位移参数。配置以 ScriptableObject 资产持久化，运行时读取资产驱动动画播放与攻击判定，形成“编辑器配置→运行时执行”的链路；工具对动画片段未赋值、时间区间非法等错误给出提示。

2.1.1 功能模块图

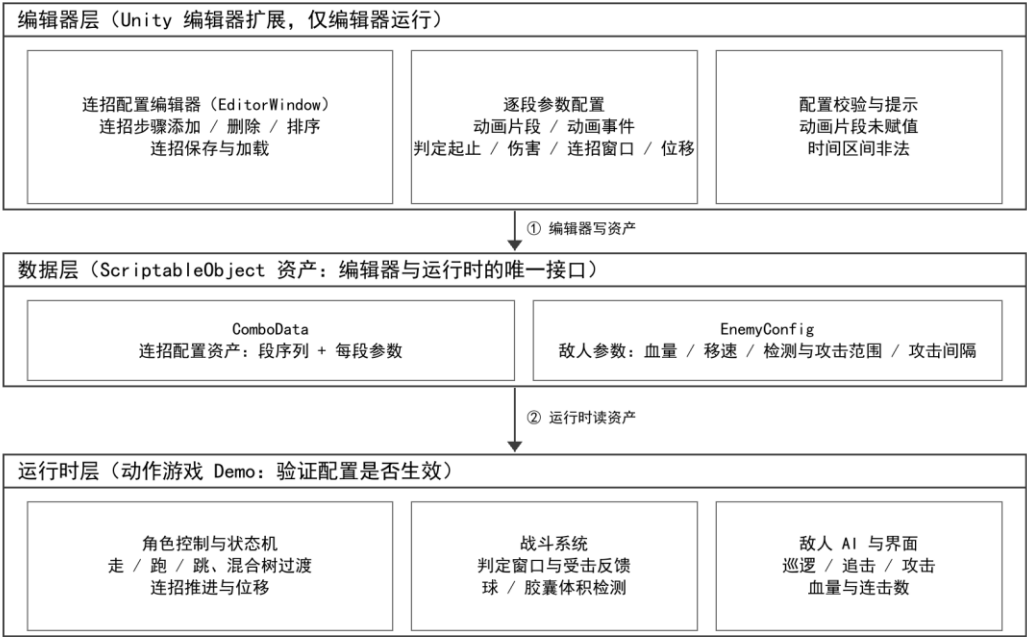


图 2-1 连招配置编辑器结构图

2.1.2 模块介绍

- （1）连招步骤与转换规则配置模块。在编辑器窗口中列出当前连招的全部步骤，支持添加、删除与排序，并配置段与段之间的转换关系：当前段可接续的下一段、连招窗口（输入下一段指令的有效时间区间）与取消窗口（允许用闪避等动作取消当前段）。
- （2）逐段参数配置模块。每一段可配置：动画片段、动画事件（该段指定时间点触发的表现事件，如打击音效）、攻击判定起始与结束时间、伤害值、连招窗口时间与位移参数（如攻击前冲距离）。
- （3）配置校验与提示模块。对动画片段未赋值、判定时间区间非法（起始晚于结束、超出动画时长）等情形给出提示，把配置错误暴露在编辑阶段。
- （4）数据持久化与运行时对接模块。配置以 ScriptableObject 资产存储，支持在编辑器内保存与加载；运行时按连招标识读取资产，驱动动画播放、判定窗口开关与位移；工具提供菜单按钮与快捷键两种重新加载入口。
- （5）角色控制与状态管理模块（验证平台）。基于 Rigidbody 实现走/跑切换与跳跃，移动动画通过 Animator 混合树平滑过渡；采用分层状态机的思路，把移动状态与攻击状态分离，通过事件通信实现状态打断（如攻击后摇可接闪避）。
- （6）战斗系统模块（验证平台）。实现不少于三段的轻攻击连招；判定窗口的开启与关闭由动画事件驱动；命中后触发受击反馈（受击动画、击退、短暂硬直）；判定采用 Physics.OverlapSphere 或胶囊体积检测，避免漏判与误判。

(7) 敌人 AI 与界面模块（验证平台）。敌人具备巡逻与追击行为，进入攻击范围后发起攻击；界面显示玩家血量与连击数。

2.1.3 预期成果

- (1) 连招配置编辑器一个，包含连招步骤管理、逐段参数配置、配置校验与提示、数据保存与加载；
- (2) 可运行于 PC 端的动作游戏 Demo 一套（角色控制与跳跃、三段以上连招、受击反馈、敌人巡逻与追击、血量与连击数界面）；
- (3) 连招配置资产与运行时对接链路，以及连招触发、命中判定、配置保存与加载的测试记录与帧率测试数据；
- (4) 毕业设计论文一篇，以及系统演示视频与答辩材料。

2.1.4 技术指标与考核标准

- (1) 开发环境：Unity 2021.3 LTS 或更高版本、URP 渲染管线、C#脚本语言。本课题实际使用团结引擎 1.9.2，其内核基于 Unity 2022.3 LTS，满足任务书对引擎版本的要求。
- (2) 编辑器工具置于 Assets/Editor 目录下，确保编辑器代码不被打包进最终构建。
- (3) 实现不少于 3 段连招的完整配置与运行，连招衔接流畅，无明显卡顿或跳帧。
- (4) 攻击判定采用 Physics.OverlapSphere 或胶囊体积检测（对应任务书要求的 OverlapSphere 与 Capsule Collider 方案），避免漏判与误判。
- (5) 连招配置编辑器具备基本的错误校验能力，如动画片段未赋值、判定时间区间非法时给出提示。
- (6) 游戏 Demo 可在 PC 端运行，帧率稳定在 60FPS 以上。

2.2 拟解决的主要问题

2.2.1 功能结构设计

系统按“编辑器层—数据层—运行时层”三层结构组织：编辑器层负责连招配置的编辑与校验；数据层以 ScriptableObject 资产承载配置，是编辑器与运行时之间唯一的接口；运行时层按资产驱动角色动画、判定与位移。层与层之间不存在直接调用，因此修改配置不需要重新编译运行时代码。

连招配置采用“段序列 + 每段参数”的数据结构：一条连招由若干有序的段组成，每段包含动画片段、动画事件、判定起止时间、伤害值、连招窗口时间与位移参数。业务规则有三条约束：连招窗口必须在动画的特定时间段内有效，超出窗口后连招中断；同一连招步骤在一次连招中不可被重复触发，直到连招结束或被重置；编辑器修改配置后，运行时通过菜单按钮或快捷键重新加载即可生效。

2.2.2 框架搭建

编辑器代码不应进入运行时构建，配置修改也应避免直接改写字段造成序列化丢失。拟采取的措施包括：编辑器脚本置于 Assets/Editor 目录并与运行时隔离；配置写入统一经由 SerializedObject 等序列化接口完成；为关键编辑操作提供撤销支持；编辑器状态通过会话状态保存。界面基于 EditorWindow 与 ImGui 搭建，用可重排列表管理连招步骤与每段参数。

2.2.3 数据库设计（数据资产层）

本课题为单机游戏客户端，不使用传统的关系数据库，本小节描述的是系统的数据资产层设计：全部可配置内容以 Unity 的 ScriptableObject 资产承载，运行时只读，编辑器统一经序列化接口写入。主要数据资产及其字段如表 1 所示。

表 1 主要数据资产及其字段

数据资产	主要字段	说明
ComboData 连招配置资产	连招段序列；每段动画片段、判定起止时间、伤害值、连招窗口时间、位移参数	本次编辑器工具的主要编辑对象，运行时据此驱动动画、判定与位移
连招段（内嵌数据）	单段参数集合（动画事件、判定区间、连招窗口、位移）	连招配置的最小配置单元
EnemyConfig 敌人配置	血量、移动速度、检测与攻击范围、攻击间隔	敌人巡逻、追击与攻击行为的参数来源

2.2.4 安全性

单机游戏的安全性主要体现在工程安全、数据正确性与运行健壮性。工程安全方面，资产修改统一经由序列化接口完成，预览与试听使用临时对象，不写入资产。

数据正确性方面，本课题在编辑期建立一组校验规则并保证与运行时语义一致，包括引用完整性、时序合法性与结构约束。这一部分是本课题的重点：数据驱动架构把错误从编译期推迟到运行期，配置错误往往不抛异常，只表现为功能不生效，定位成本高；把校验前移到编辑期可以直接降低这类风险。运行健壮性方面，连招或攻击被打断时统一清理，避免判定窗口与位移残留到后续行为。

2.2.5 功能性测试

测试分两部分。功能测试覆盖游戏侧与工具侧：游戏侧包括连招触发与推进（连续输入是否进入下一段、窗口外输入是否被忽略、同一步骤是否重复触发、超出窗口是否正确中断）、命中判定、受击反馈与敌人 AI 状态转移；工具侧包括连招

步骤增删排序、配置保存与加载、校验提示是否触发，以及重新加载配置后运行时能否立即生效。

自动化与指标测试：EditMode 单元测试覆盖数据层与连招推进逻辑；命令行编译门禁保证提交代码可编译；使用 Profiler 统计 PC 端帧率，验证是否稳定在 60FPS 以上。

2.2.6 其他关键技术问题

动画事件与判定窗口的同步（触发时机与配置时间不一致会导致判定错位）；连招窗口的手感调校（窗口过窄导致连招接不上、过宽导致乱按也能连）；配置改动后的运行时重新加载策略；以及高频对象的复用与状态重置。

三、研究步骤、方法及措施

3.1 研究步骤

（1）需求分析与资料收集：查阅任务书、引擎官方文档与编辑器开发资料，明确连招配置工具的功能边界与配置对象范围。本阶段已完成。

（2）动作游戏 Demo 核心闭环：实现角色控制、攻击连招、受击反馈、敌人状态机与胜负结算，为工具提供真实的配置对象与验证场景。本阶段已完成。

（3）数据层与编辑器工具主体：定义连招配置的数据资产，实现编辑器窗口、参数配置界面与配置校验功能。本阶段已完成主体，正在按任务书口径扩展连招配置能力。

（4）连招配置编辑器完善与补充功能：实现连招步骤的添加、删除与排序，补齐每段动画片段、动画事件、判定起止时间、伤害值、连招窗口时间与位移参数的配置入口；补充角色跳跃、敌人巡逻与连击数界面。本阶段正在进行。

（5）校验、测试与论文撰写：完善配置校验规则，开展连招触发、命中判定、配置保存加载与帧率测试；论文按任务书给定的六章结构（绪论、相关技术、系统分析、系统设计、系统实现、系统测试、结论）组织。本阶段计划进行。

3.2 研究方法

（1）文献与资料研究法。以任务书、Unity 官方文档与编辑器开发专著为依据，确定设计取舍。

（2）原型迭代法。先打通“配置数据—运行时读取—表现反馈”的最小闭环，再逐步扩展连招段数与配置项，每一步都在运行环境中验证。

（3）对照实验与测试法。对连招参数使用编辑器配置与直接修改代码的人工方式作对照，比较修改步骤数与出错概率；对数据层与连招推进逻辑编写 EditMode 自动化测试，并使用 Profiler 验证帧率指标。

3.3 技术路线与措施

系统的数据流为：连招配置编辑器（编辑连招步骤与每段参数，写入资产）→ 数据层（连招配置、敌人参数等 ScriptableObject 资产）→ 运行时（读取资产，驱动动画播放、判定窗口开关与位移）。编辑器层与运行时层之间没有直接调用，唯一接口是资产本身。主要措施：编辑器脚本统一置于 Assets/Editor；使用 Git 管理版本；以命令行编译检查与单元测试作为提交门禁；对关键配置项建立校验规则；用 Profiler 定位性能瓶颈。

四、研究工作进度

本课题起止日期为 2026 年 6 月 10 日至 2027 年 1 月 12 日，进度安排与任务书一致，见下表。

序号	时间	内容
1	2026.06.10 - 2026.06.19	课题申报
2	2026.06.20 - 2026.06.23	学生选题
3	2026.06.24 - 2026.06.26	确认选题
4	2026.06.27 - 2026.07.06	任务书
5	2026.07.07 - 2026.09.21	开题答辩
6	2026.09.22 - 2026.11.18	软件编码、论文撰写
7	2026.10.21 - 2026.10.27	中期检查
8	2026.11.19 - 2026.11.25	院内互评
9	2026.11.30 - 2026.12.4	校内评阅
10	2026.12.7 - 2026.12.18	校外盲审
11	2026.12.22 - 2026.12.31	毕业答辩
12	2027.01.03 - 2027.01.12	成绩评议，材料归档

五、主要参考文献

[1] 张寿昆. Unity 编辑器开发与拓展[M]. 北京: 清华大学出版社, 2024.

[2] NYSTROM R. 游戏编程模式[M]. 北京: 人民邮电出版社, 2016.

[3] Gordon. ACT 技能编辑器的制作经验分享[EB/OL]. 侑虎科技, 2018.

[4] Unity Technologies. Introduction to editor scripting[EB/OL]. Unity Learn, 2025.

[5] Unity Technologies. Unity manual: scripting concepts[EB/OL]. Unity Documentation, 2025.

[6] GREGORY J. Game engine architecture[M]. 3rd ed. Boca Raton: CRC Press, 2018.

[7] 王巍, 高德远. 有限状态机设计策略[J]. 计算机工程与应用, 1999(7): 54-55.

[8] 胡凯宁. 基于 Unity3D 的像素休闲游戏的设计与实现[J]. 电脑编程技巧与维护, 2026(4): 147-149.